Hashiko

Distributed File Integrity Monitoring System with Temporal Correlation

Jackie Ma | A00889988
Applied Computer Science
British Columbia Institute of Technology
Fall 2025

Presentation Objectives

Introduction
Background & Problem Statement
Prototype
Milestone 1
Milestone 2
System Architecture
Technical Challenges
Limitations
Future Work
Conclusion

Introduction

Key Components:

- Endpoint agents
- Central Flask server
- PostgreSQL database
- Web interface

Core Features:

- Centralized monitoring
- Queue-based retry logic
- Cross-platform support
- Timestamped events

Background & Problem Statement

Traditional FIM Challenges:

- Local tools can be disabled by attackers with admin privileges
- Detection delays in periodic checking
- Performance vs detection trade-off
- Manual policy configuration overhead

Solution:

- Centralized monitoring prevents local compromise
- Timestamped hash collection enables temporal analysis
- Network-resilient architecture with queue system
- Simplified deployment across distributed systems

Prototype

Agent Features

- SHA-256 hash directories (/boot, /usr/bin, /usr/sbin, /etc and /root)
- Path filtering (cache, temp)
- JSON output with timestamps

Server Features

- Go net/http server
- PostgreSQL storage
- /api/store endpoint for ingest
- /api/view for record retrieval

Key Achievement: Basic agent-server communication with file integrity monitoring and parameterized SQL queries to prevent injection attacks

Milestone 1: Backend Rewrite

Migration to Python/Flask

- Simplified server management
- Better ecosystem integration
- Easier maintenance
- Retained Go agents for performance

Database Optimization

- Composite indexes added (timestamp, agent_id, event_type)
- Fast multi-dimensional queries
- Optimized column order

Performance Impact: Query response time improved from seconds to near instantaneously

Milestone 2: Network Resilience

Problem Solved

- Network interruptions caused data loss and monitoring gaps

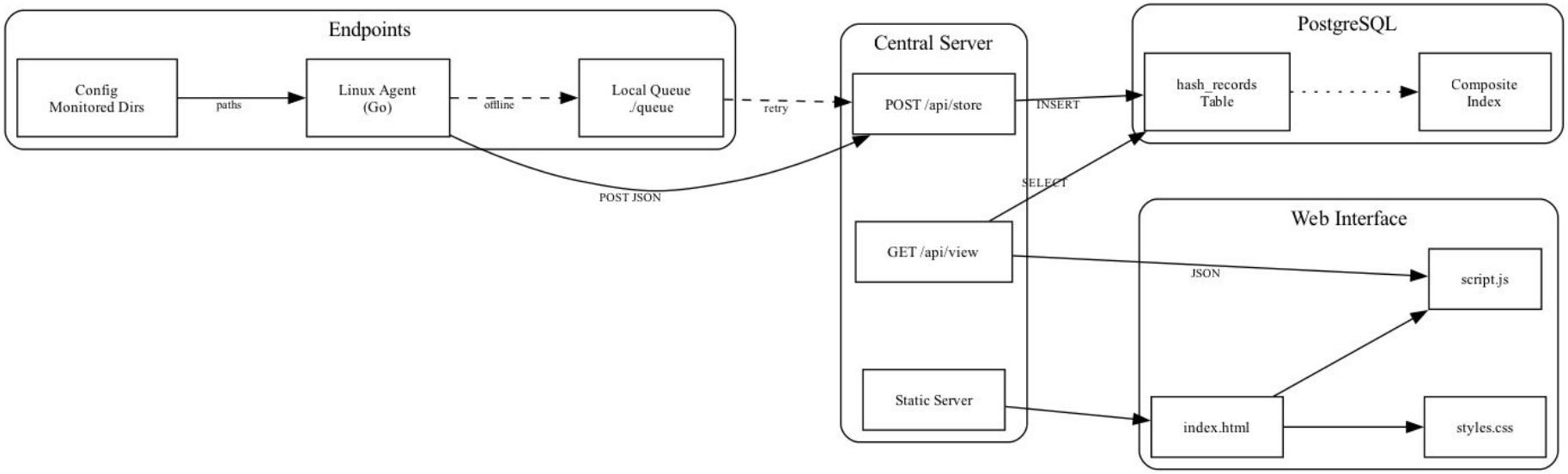
Implementation

- Queue directory initialization
- Save failed transmissions as timestamped JSON files
- Automatic retry on reconnection
- Remove files after successful transmission

Result

- Zero data loss during network outages
- All three queued events delivered after reconnection in integration tests

System Architecture



Technical Challenges

Challenges	Solutions
Large directory trees with thousands of files slow agent startup (/boot, /usr/bin, /usr/sbin, /etc and /root)	Path filtering (cache, temp files);
Must support fast queries across timestamp, agent_id, and event_type	Composite indexes (timestamp, agent_id and event_type)
Network interruptions cause data loss when agents cannot reach server	Queue system saves JSON to local files; retries when connection restored

Limitations

Limited scope: Excludes login monitoring and process monitoring

Basic UI: Simple table display without advanced filtering

Platform support: Linux

Future Work

Milestone 3: Advanced Detection

- Temporal correlation for multi-step attack detection
- Rule-based detection engine with configurable policies
- Pattern matching for attack signatures

Milestone 4: Real-Time Alerting

- Web interface notification system
- Immediate alerts for suspicious changes
- Configurable alert thresholds and severity levels

Long-Term Vision: Machine learning anomaly detection, enhanced web interface with advanced filtering and macOS/Windows agent support

Conclusion

COMP8800 Achievements

✓ Centralized file integrity monitoring across distributed systems

✓ Network-resilient architecture with zero data loss

Foundation Complete

Reliable agent-server architecture with robust data collection, queue-based retry logic, and optimized storage ready for advanced features

Next Phase (COMP8900)

Build on this foundation with temporal correlation, rule-based detection, and real-time alerting capabilities

Q&A